JOMHACK
EMPOWERING INNOVATION

```
> bootcamp.learn()
```

JOMHACK
EMPOWERING INNOVATION

JomCode Coding Bootcamp

CHAPTER 08

Modules, packages, objects and type annotations

```
learn / type / run / explain
```

Plain meaning and technical explanation

PLAIN MEANING

Modules organize code. Objects keep data together with behavior.

TECHNICAL EXPLANATION

import loads a module. Packages group modules. Classes define object behavior. Type annotations describe intended types but do not enforce validation alone.

REAL-WORLD USE

Read settings objects, SDK clients and typed service functions in Postlight.

QUICK TIP

Know the difference between installing a package and importing its module.

INSTALL / RUN

Standard library only. Windows: py workbook.py. macOS: python3 workbook.py.

Worked example 1: Import a standard module

workbook.py

```
from pathlib import Path
path = Path("posts") / "draft.json"
print(path.name)
print(path.suffix)
```

PRO TIP Path handles Windows and macOS separators.

Run and explain example 1

EXPECTED RESULT

draft.json
.json

WHY IT BEHAVES THIS WAY

Path is an imported class. The / operator joins path components.

Worked example 2: A class and an instance

workbook.py

```
class Post:
    def __init__(self, title: str):
        self.title = title.strip()
    def preview(self) -> str:
        return self.title[:5]

post = Post(" Hello world ")
print(post.preview())
```

PRO TIP A method call such as client.auth.get_user() follows the same object pattern.

JOMHACK
EMPOWERING INNOVATION

Run and explain example 2

EXPECTED RESULT

Hello

WHY IT BEHAVES THIS WAY

__init__ initializes the instance. self refers to the instance receiving the method call.

Worked example 3: Annotations and optional values

workbook.py

```
01    def image_label(path: str | None) -> str:
02        return "No image" if path is None else path
03
04    print(image_label(None))
05    print(image_label("a/photo.png"))
```

PRO TIP Pydantic later uses annotations to validate incoming data.

Run and explain example 3

EXPECTED RESULT

No image
a/photo.png

WHY IT BEHAVES THIS WAY

The union annotation permits a string or None. The function still contains the behavior.

Worked example 4: Qualified names and entry points

workbook.py

```
import json
def main():
    print(json.dumps({"ok": True}))

if __name__ == "__main__":
    main()
```

PRO TIP Project modules use imports such as from postlight.config import settings.

Run and explain example 4

EXPECTED RESULT

```
{"ok": true}
```

WHY IT BEHAVES THIS WAY

The entry-point guard runs main() only when this file is executed directly.

Exercise 1: Typed summary

YOUR TASK

Create a typed function that accepts a post dictionary and returns its trimmed title.

CHECK THESE CASES

" Hi " becomes Hi. Input dictionary remains unchanged.

HINT AFTER A TRY

Use dict[str, str] and return a new string.

Exercise 2: Client-shaped object

YOUR TASK

Create a Counter class with count initially zero and an increment method.

CHECK THESE CASES

Two instances maintain separate counts. Incrementing one leaves the other at zero.

HINT AFTER A TRY

Store count on self.

Checkpoint before moving on

01 Run and explain all four worked examples. Match their actual results to the expected behavior.

02 Attempt both exercises before reading the solutions. Verify every acceptance case.

03 Explain how this chapter supports Postlight: Read settings objects, SDK clients and typed service functions in Postlight.

Official reference